

Ejercicios de repaso — Angular y Spring Boot

Nivel orientado a 2º de Desarrollo de Aplicaciones Web

Documento de práctica con ejercicios progresivos para repasar Angular y Spring Boot, incluyendo fundamentos, formularios, rutas, servicios, APIs REST, JPA, validación y pequeños proyectos integrados.

Angular

Bloque 1. Fundamentos

Ejercicio 1 — Componente básico

- Crear un componente **saludo** que muestre tu nombre, tu curso y un mensaje de bienvenida.
- Practicar la creación de componentes y la interpolación con `{{ }}`.

Ejercicio 2 — Property binding

- Crear un componente con un título, una imagen y un enlace.
- Usar `[src]`, `[href]` y `[alt]`.

Ejercicio 3 — Event binding

- Crear un botón que al pulsarlo muestre un mensaje en consola y cambie el texto de un párrafo.
- Usar `(click)`.

Ejercicio 4 — Two-way binding

- Crear un formulario con un campo de texto para introducir el nombre del usuario.
- Debajo debe aparecer: **Hola, [nombre introducido]**.
- Usar `[(ngModel)]`.

Bloque 2. Directivas

Ejercicio 5 — `ngIf`

- Crear un botón para mostrar u ocultar un texto usando `*ngIf`.

Ejercicio 6 — `ngFor`

- Crear un array con 5 módulos de 2º DAW y mostrarlos en una lista usando `*ngFor`.

Ejercicio 7 — `ngClass` y `ngStyle`

- Crear una lista de alumnos con notas.
- Si la nota es mayor o igual que 5, mostrar en verde; si es menor, en rojo.
- Usar `[ngClass]` o `[ngStyle]`.

Bloque 3. Componentes y comunicación

Ejercicio 8 — `@Input`

- Crear un componente padre y un componente hijo.
- El padre enviará al hijo nombre de producto, precio y stock.
- El hijo deberá mostrar esa información usando `@Input`.

Ejercicio 9 — @Output

- Crear un componente hijo con un botón **Eliminar**.
- Al pulsarlo, debe avisar al padre para que muestre en consola qué elemento se ha eliminado.
- Usar **@Output** y **EventEmitter**.

Bloque 4. Formularios

Ejercicio 10 — Formulario template-driven

- Crear un formulario de registro con nombre, email, contraseña y edad.
- Validar nombre obligatorio, email obligatorio, contraseña mínima de 6 caracteres y edad mínima de 18 años.
- Mostrar mensajes de error.

Ejercicio 11 — Formulario reactivo

- Repetir el ejercicio anterior usando **FormGroup**, **FormControl** y **Validators**.

Ejercicio 12 — Validación personalizada

- Añadir un campo **confirmar contraseña** en un formulario reactivo.
- Validar que la contraseña y su confirmación coincidan.

Bloque 5. Servicios y consumo de API

Ejercicio 13 — Servicio simple

- Crear un servicio **usuarios.service.ts** que devuelva un array local de usuarios con id, nombre y email.
- Mostrar los usuarios en un componente.

Ejercicio 14 — HttpClient

- Consumir una API pública y mostrar resultados en un componente.
- Practicar **HttpClient** y **subscribe**.

Ejercicio 15 — Listado y detalle

- Mostrar una lista de productos y, al pulsar sobre uno, mostrar su detalle.
- Practicar rutas, parámetros en ruta y navegación.

Bloque 6. Rutas

Ejercicio 16 — Routing básico

- Crear una aplicación con las rutas **/inicio**, **/productos** y **/contacto**.
- Añadir un menú de navegación.

Ejercicio 17 — Ruta con parámetro

- Crear una ruta **/alumno/:id** y mostrar un alumno distinto según el id.
- Usar **ActivatedRoute**.

Ejercicio 18 — Ruta protegida

- Crear una ruta **/admin** protegida con un guard.
- Solo debe entrar si una variable **logueado** vale **true**.

Bloque 7. CRUD en Angular

Ejercicio 19 — CRUD local

- Crear una aplicación de tareas que permita listar, añadir, editar y eliminar tareas.
- Usar datos locales, sin backend.

Ejercicio 20 — CRUD con backend

- Conectar Angular a una API REST y permitir ver, crear, editar y borrar productos.

Bloque 8. Ejercicios más completos

Ejercicio 21 — Mini agenda

- Crear una agenda de contactos con listado, búsqueda por nombre, alta y eliminación.

Ejercicio 22 — Tienda simple

- Crear una tienda con catálogo, detalle de producto, carrito y total de compra.

Ejercicio 23 — Panel de alumnos

- Crear una aplicación para listar alumnos, filtrar por aprobado o suspenso, ordenar por nota y ver el detalle de cada alumno.

Spring Boot

Bloque 1. Fundamentos

Ejercicio 1 — Proyecto básico

- Crear un proyecto Spring Boot con dependencia web.
- Crear la ruta **/hola** que devuelva **Hola desde Spring Boot**.

Ejercicio 2 — Controlador con varias rutas

- Crear un controlador con las rutas **/saludo**, **/despedida** y **/curso**, cada una con un texto distinto.

Ejercicio 3 — Parámetros en URL

- Crear la ruta **/saludar?nombre=Fernando** que devuelva **Hola Fernando**.
- Usar **@RequestParam**.

Ejercicio 4 — PathVariable

- Crear la ruta **/producto/5** que muestre **Producto con id 5**.
- Usar **@PathVariable**.

Bloque 2. MVC básico

Ejercicio 5 — Controlador + vista

- Si trabajáis con Thymeleaf o MVC clásico, crear una vista HTML que muestre un mensaje enviado desde un controlador.
- Practicar **Model** y retorno de vistas.

Ejercicio 6 — Formulario

- Crear un formulario con nombre, apellidos y edad.
- Al enviarlo, mostrar otra vista con los datos recibidos.

Bloque 3. Capas

Ejercicio 7 — Clase entidad simple

- Crear una clase **Producto** con id, nombre, precio y stock.
- Crear un controlador que devuelva varios productos en formato JSON.

Ejercicio 8 — Separación por capas

- Organizar un proyecto con **controller**, **service** y **model**.
- El controlador debe llamar al servicio y el servicio devolver una lista de productos.

Bloque 4. API REST

Ejercicio 9 — GET de lista

- Crear el endpoint **GET /productos** que devuelva una lista de productos.

Ejercicio 10 — GET por id

- Añadir **GET /productos/{id}** para devolver un producto concreto.
- Si no existe, mostrar un mensaje adecuado.

Ejercicio 11 — POST

- Añadir **POST /productos** para crear un producto enviando JSON.

Ejercicio 12 — PUT

- Añadir **PUT /productos/{id}** para actualizar nombre, precio y stock.

Ejercicio 13 — DELETE

- Añadir **DELETE /productos/{id}** para eliminar un producto.

Bloque 5. JPA y base de datos

Ejercicio 14 — Entidad JPA

- Configurar la entidad **Producto** con **@Entity**, **@Id** y **@GeneratedValue**.

Ejercicio 15 — Repository

- Crear **ProductoRepository extends JpaRepository<Producto, Long>** y usarlo para listar productos desde la base de datos.

Ejercicio 16 — CRUD completo con BD

- Conectar Spring Boot con MySQL y hacer un CRUD completo sobre la tabla **productos** con id, nombre, precio y stock.

Ejercicio 17 — Búsqueda personalizada

- Añadir un método en el repositorio para buscar productos por nombre.
- Ejemplo: productos que contengan **portátil**.

Bloque 6. Relaciones

Ejercicio 18 — Relación uno a muchos

- Crear las entidades **Categoria** y **Producto**.
- Una categoría tiene muchos productos y un producto pertenece a una categoría.
- Usar **@OneToMany** y **@ManyToOne**.

Ejercicio 19 — Relación muchos a muchos

- Crear las entidades **Alumno** y **Asignatura** con relación muchos a muchos.

Bloque 7. Validación

Ejercicio 20 — Validación de datos

- Validar en la API de productos que el nombre no esté vacío, que el precio sea mayor que 0 y que el stock no sea negativo.
- Usar **@Valid** y anotaciones de validación.

Bloque 8. Ejercicios completos

Ejercicio 21 — API de biblioteca

- Crear una API REST para gestionar libros con id, título, autor, género y ejemplares.
- Permitir listar, ver por id, añadir, editar y borrar.

Ejercicio 22 — API de alumnos

- Crear una API REST con alumnos y cursos.
- Permitir listar alumnos, filtrar por curso, añadir y borrar.

Ejercicio 23 — Tienda con categorías

- Crear una API para una tienda con productos y categorías.
- Permitir CRUD de productos, listado por categoría, validación y conexión a MySQL.

Ejercicios mixtos Angular + Spring Boot

- CRUD completo de productos: backend en Spring Boot y frontend en Angular con listado, alta, edición y borrado.
- Gestión de tareas: backend con endpoints de tareas y frontend para ver, crear, completar y eliminar tareas.
- Biblioteca: backend con libros y autores, y frontend con listado, filtro por género, detalle y formulario de alta.

Propuesta de repaso

Para un repaso ordenado en Angular conviene seguir esta secuencia: componentes, directivas, formularios, servicios, rutas y CRUD. En Spring Boot resulta útil avanzar por rutas simples, controladores REST, capa service, repositorios, JPA, relaciones, validaciones y CRUD completo.

Tres ejercicios tipo examen

- Angular: aplicación de gestión de películas con listado, filtro por género, alta, eliminación y vista detalle.
- Spring Boot: API REST de empleados con id, nombre, departamento y salario; CRUD completo, validación y MySQL.
- Angular + Spring Boot: aplicación de alumnos con listado, alta, edición, borrado y búsqueda por nombre.

Todos los ejercicios están planteados con una dificultad coherente con 2º DAW y sirven tanto para practicar de forma aislada como para convertirlos en miniproyectos de entrega.